

W E E K 1 4

데이터 전처리와 과적합 대응

Data Preprocessing & Overfitting

기계학습 라이브러리 활용

P A R T 1

데이터 전처리

Data Preprocessing

결측치

수치형
정규화

범주형
인코딩

시간
변수

3분할

1.1 결측치 처리

탐지

```
# 각 컬럼별 결측치 개수
print(df.isnull().sum())
```

```
age      1
salary   1
city      1
```

처리 전략 3가지

방법	쓰임
<code>dropna()</code>	결측 < 5%
<code>fillna(mean)</code>	수치, 정규
<code>fillna(median)</code>	수치, 치우침
<code>fillna('Unknown')</code>	범주형

WARNING — Data Leakage 방지

대체값(평균, 중앙값)은 **train 데이터에서만** 계산해야 한다.
test 데이터의 평균을 쓰면 미래 정보가 학습에 누수됨 (data leakage)

1.2 수치형 정규화

왜 정규화하나?

변수 A: [1, 2, 3, 4]

변수 B: [100, 200, 300, 400]

→ 신경망은 큰 값에 과민 반응

→ 학습 불안정

두 가지 방법

Scaler	Range	When
MinMax	[0, 1]	image px
Standard	$\mu=0, \sigma=1$	normal-ish

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled   = scaler.transform(X_val)   # fit X
X_test_scaled  = scaler.transform(X_test)  # fit X
```

1.3 범주형 인코딩

(a) One-Hot

순서 없는 범주 (*color, city*)

```
pd.get_dummies(df,
               columns=['color'])

# color_red  color_blue
#         1         0
#         0         1
```

주변수가 많아지지만 가장 안전

(b) Ordinal

순서 있는 범주 (*size, grade*)

```
size_map = {
    'S': 0, 'M': 1,
    'L': 2, 'XL': 3
}
df['size'] = df['size'].map(size_map)
```

순서를 모델에 알려줌

1.4 시간 변수 처리

datetime을 수치형 feature로 분해

```
df['date'] = pd.to_datetime(df['date'])

df['year']      = df['date'].dt.year
df['month']     = df['date'].dt.month      # 1 ~ 12
df['day']       = df['date'].dt.day        # 1 ~ 31
df['hour']      = df['date'].dt.hour       # 0 ~ 23
df['dayofweek'] = df['date'].dt.dayofweek  # 0=Mon, 6=Sun
df['is_weekend'] = (df['dayofweek'] >= 5).astype(int)
```

TIP 시간은 **순환적**(23시 다음은 0시). 더 정교하게 표현하려면 sin/cos 변환 사용

1.5 Train / Val / Test 3분할

왜 3분할인가?

Train → 가중치 학습

Val → 하이퍼파라미터 결정 + 과적합 진단

Test → 마지막 단 한 번만 평가

2분할의 함정: test로 튜닝하면

test가 '새는' 샘플 → 최종 평가 불가능

```
from sklearn.model_selection import train_test_split

# 1) train+val vs test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2) train vs val
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.2, random_state=42)

# 64% train / 16% val / 20% test
```

Keras에서:

```
model.fit(X_train, y_train,
          validation_data=(X_val, y_val), ...)
```

Part 1 정리

- 1 **결측치** 탐지 → 삭제 또는 대체. train에서만 통계 계산
- 2 **수치형** StandardScaler 기본. train에 fit, val/test는 transform
- 3 **범주형** 순서 있으면 Ordinal, 없으면 One-Hot
- 4 **시간** datetime → hour, dayofweek, month 분해
- 5 **3분할** Train(학습) / Val(튜닝) / Test(최종 평가)

Week 13까지는 #2와 2분할만 다뤘다 — 오늘부터 5가지 모두가 표준

P A R T 2

과적합 진단

Overfitting Diagnosis

"Train loss는 떨어지는데 Val loss가 오르면 → 과적합"

2.1 과적합이란

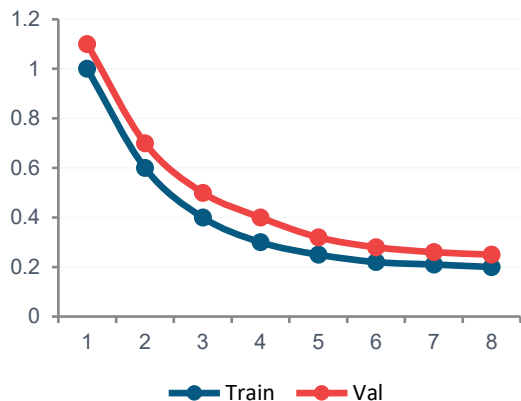
좋은 모델 : train에서 잘 맞추고, 새 데이터에서도 잘 맞춤

과적합 : train에서만 잘 맞추고, 새 데이터에서는 못 맞춤

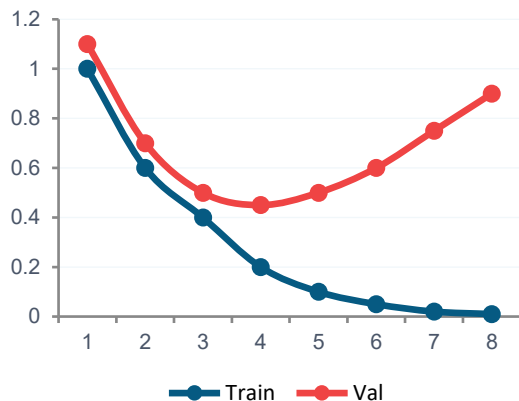
상태	Train Loss	Val Loss	진단 / 처방
Underfit	높음	높음	모델 너무 단순 → 모델 키우기
Good fit	낮음	낮음	적절
Overfit	매우 낮음	높음	모델 너무 복잡 → 정규화

2.2 Loss Curve로 진단

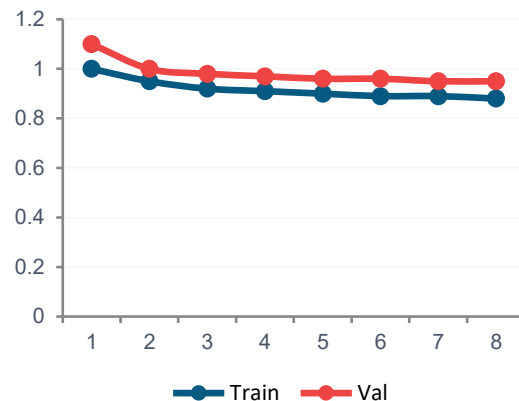
Good Fit



Overfit



Underfit



과적합 시그널: Train loss 계속 감소 · Val loss 어느 시점부터 **상승** · 둘 사이 **갭이 벌어짐**

2.3 과적합을 일부러 만들어보기

작은 데이터 + 큰 모델 = 과적합 레시피

```
X_train_small = X_train[:1000]      # 데이터를 일부러 줄임

model = keras.Sequential([
    keras.layers.Dense(256, activation='relu', input_shape=(n,)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(256, activation='relu'),      # 모델은 큼
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(1)
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
history = model.fit(X_train_small, y_train_small,
                    validation_data=(X_val, y_val), epochs=100, batch_size=32)
```

TIP 실험 습관

새 데이터를 만나면 과적합을 일부러 한 번 만들어본다. 최대 표현력을 확인한 후 정규화로 줄여나가는 게 안전.

P A R T 3

정규화

Regularization Toolkit

Dropout

무작위 비활성

L2

가중치 페널티

Early Stop

최선 시점 정지

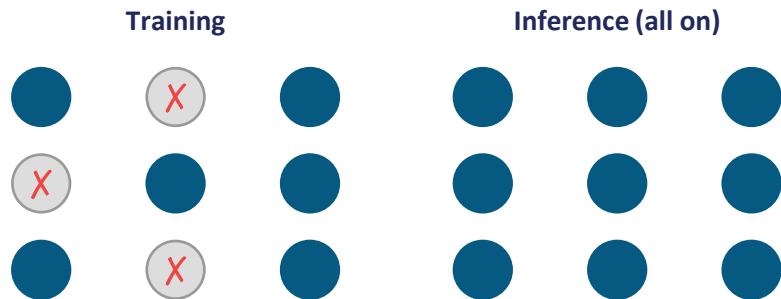
BatchNorm

층 입력 정규화

Early Stopping 이 가장 실용적 — 항상 먼저 적용

3.1 Dropout

직관: 매 step마다 일부 뉴런을 꺼버린다



효과: 앙상블 + 공의존 방지

```
from keras.layers import Dropout

model = keras.Sequential([
    Dense(128, activation='relu',
    input_shape=(n,)),
    Dropout(0.3),           # 30% off
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(1)
])
```

rate: 0.2 ~ 0.5 · 출력층 직전엔 사용 안 함

3.2 L2 Regularization

직관: Loss에 가중치 페널티 추가

원래 : $L = \text{MSE}$

L2 : $L = \text{MSE} + \lambda \cdot \sum w_i^2$

가중치가 클수록 페널티 ↑
 → 모델이 작은 가중치 선호
 → 단순한 함수 학습
 → 과적합 ↓

```
from keras.regularizers import l2

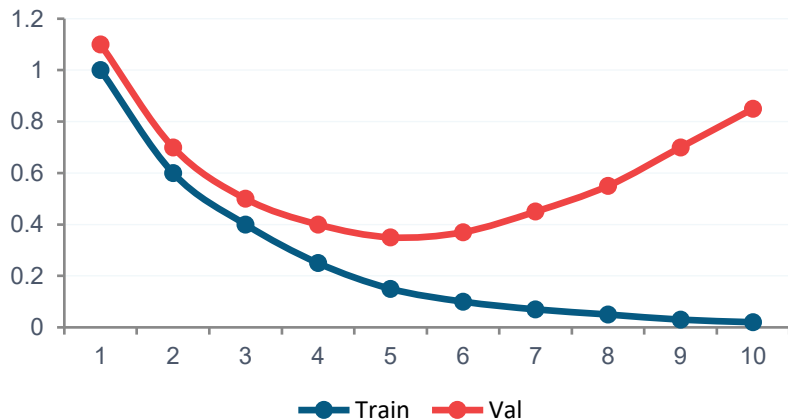
reg = l2(0.01)      # λ = 0.01

model = keras.Sequential([
    Dense(128, activation='relu',
          kernel_regularizer=reg,
          input_shape=(n,)),
    Dense(128, activation='relu',
          kernel_regularizer=reg),
    Dense(64, activation='relu',
          kernel_regularizer=reg),
    Dense(1)
])
```

L2 (Ridge): $\sum w_i^2 \rightarrow$ 가중치를 작게 · **L1 (Lasso):** $\sum |w_i| \rightarrow$ 정확히 0으로 (feature 선택) · **신경망은 보통 L2**

3.3 Early Stopping (가장 실용적)

직관: Val loss가 개선 안 되면 멈춤



★ 여기서 멈춰라(epoch 5)

```
from keras.callbacks import EarlyStopping

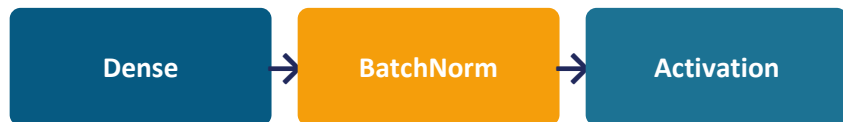
es = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

model.fit(X_train, y_train,
          validation_data=(X_val, y_val),
          epochs=200,           # 크게 잡아도 OK
          callbacks=[es])
```

왜 가장 실용적? 하이퍼파라미터 거의 없음 · epochs 크게 잡아도 안전 · 다른 정규화와 조합 가능

3.4 Batch Normalization

직관: 각 층의 입력을 미니배치 단위로 정규화



효과

- 학습 안정성 ↑
- 큰 learning rate 사용 가능
- 미니배치 noise → 약한 정규화
- 가중치 초기값 민감도 ↓

```
from keras.layers import (  
    BatchNormalization, Activation)  
  
model = keras.Sequential([  
    Dense(128, input_shape=(n,)), # no act  
    BatchNormalization(),  
    Activation('relu'),  
    Dense(128),  
    BatchNormalization(),  
    Activation('relu'),  
    Dense(1)  
])
```

순서 중요 Dense → BN → Activation

3.5 조합 가이드

실용적인 적용 순서

- 1 Early Stopping (항상)
- 2 Loss curve 확인
- 3 여전히 과적합 → Dropout 또는 L2 추가
- 4 학습이 불안정 → BatchNormalization 추가

주의 Dropout + BatchNorm 동시 사용

학계에 논쟁 있음 (서로 간섭 가능). 둘 다 쓰려면 **BN → Activation → Dropout** 순서.
보통은 둘 중 하나만 써도 충분.

Part 3 정리

방법	핵심 코드	효과
Dropout	Dropout(0.3)	뉴런 무작위 비활성 → 앙상블 효과
L2	kernel_regularizer=l2(0.01)	가중치 작게 → 단순한 함수
Early Stopping	EarlyStopping(patience=10)	최선 시점 자동 정지 (우선 적용)
BatchNorm	BatchNormalization()	층 입력 정규화 → 학습 안정

한 줄로: Early Stopping은 항상, 나머지는 필요할 때

Part 4 · 종합 실습 시나리오

데이터: UCI Appliances Energy Prediction

샘플 19,735 / 26 features / 실내·실외 기상 / 회귀 (Wh)

```
url = 'https://raw.../energydata_complete.csv'
df = pd.read_csv(url)
df['date'] = pd.to_datetime(df['date'])

# 시간 변수 추출 (1.4)
df['hour'] = df['date'].dt.hour
df['dayofweek'] = df['date'].dt.dayofweek
df['month'] = df['date'].dt.month

X = df[feature_cols].values
y = np.log1p(df['Appliances'].values) # long-tail → log

# 3분할 (1.5)
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.2)

# Scaling (1.2)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val, X_test = scaler.transform(X_val), scaler.transform(X_test)
```

Baseline vs Regularized

Baseline · no regularization

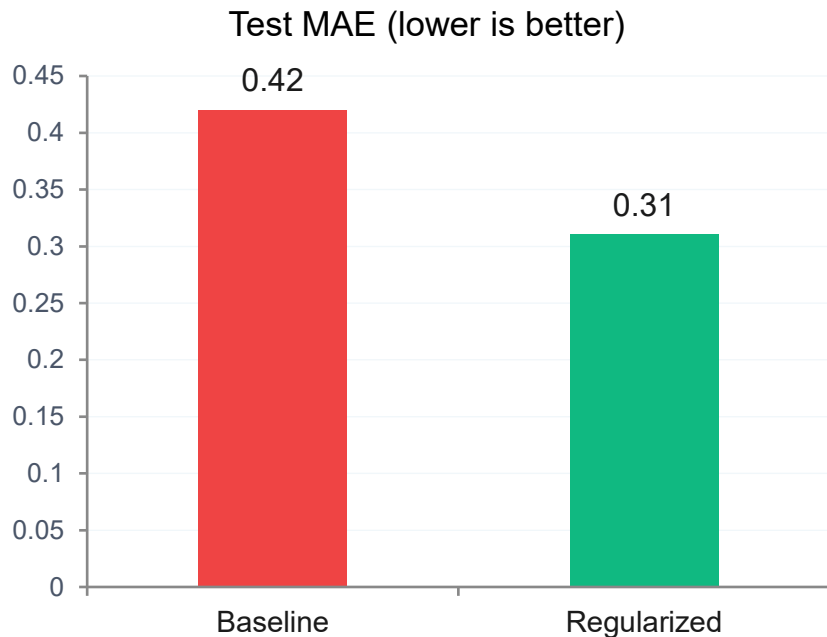
```
baseline = keras.Sequential([
    Dense(128, activation='relu',
          input_shape=(n,)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(1)
])
baseline.compile(optimizer='adam',
                  loss='mse',
                  metrics=['mae'])

baseline.fit(X_train, y_train,
             validation_data=(X_val, y_val),
             epochs=100, batch_size=32)
```

Regularized

```
regularized = keras.Sequential([
    Dense(128, kernel_regularizer=l2(.001)),
    BatchNormalization(),
    Activation('relu'),
    Dropout(0.3),
    Dense(128, kernel_regularizer=l2(.001)),
    BatchNormalization(),
    Activation('relu'),
    Dropout(0.3),
    Dense(1)
])
es = EarlyStopping(patience=15,
                    restore_best_weights=True)
regularized.fit(..., callbacks=[es])
```

결과 비교 (예상)



항목	Baseline	Regularized
Train loss	매우 낮음	약간 높음
Val loss	어느 시점부터 ↑	안정적
Train-Val gap	큼	작음
Test MAE	나쁨	좋음

결론

정규화는 train 성능을 약간 희생,
val/test 성능을 크게 개선